Reconsider parallel `ranges::rotate_copy` and `ranges::reverse_copy`

Document #:	P3709R2 [Latest] [Status]
Date:	2025-06-18
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Ruslan Arutyunyan <ruslan.arutyunyan@intel.com> Alexey Kukanov <alexey.kukanov@intel.com>

Contents

§ Abstract

This paper proposes changing the return value type for parallel `ranges::rotate_copy` and `ranges::reverse_copy` in [P3179R9].

§ 1 Introduction

LEWG approved the "range-as-the-output" design aspect for our parallel range algorithms proposal [P3179R9]. It introduces the use of a range with its own bound as the output sequence, taking into account that the output size may be less than the input size. A common questions for such algorithms is which iterator to return for the input when the output size is not sufficient to hold the processed data?

For most cases the answer is pretty simple: we return the iterator which points to the position right after the last processed element, or *one past the last*. Consider `std::ranges::copy` as an example:

```
std::vector input{1,2,3,4,5};
std::vector<int> output(5); // output with sufficient size
std::vector<int> smaller_output(3); // output with insufficient size

auto res1 = std::ranges::copy(std::execution::par, input, output);

// after copy invocation res1.in == input.end() is true

auto res2 = std::ranges::copy(std::execution::par, input, smaller_output);

// after copy invocation res2.in == input.begin() + 3 is true
// res2.in points to the element equal to 4
```

It gives us consistent behavior with serial range algorithms when the output size is sufficient, and returns the *stop point* in the input when the output size is insufficient.

The proposed design in [P3179R9] follows the same logic (returning the stop point for the input) for parallel `ranges::rotate_copy` and `ranges::reverse_copy`, and both algorithms use the same return type as their serial range counterparts. It's consistent with the rest of the parallel range algorithms that have an output. However, it leads to interesting consequences:

— `reverse_copy` goes in the reverse order, so the one past the last iterator for the input never equals to `last`, unless the output range is empty.

— `rotate_copy` goes over two subranges within the input range, starting from `middle`. The one past the last iterator for the input also never equals to `last`: it's one of the iterators in either `[middle, last - 1]` or `[first, middle]`.

The proposed design in [P3179R9] allows users to know where the algorithm stops if the output size is not sufficient to hold all the elements from the input. The potential problem, however, comes from the combinations of three factors:

— in that design, the return type for serial and parallel range algorithms is the same.

— serial range algorithms always return `last` for both `reverse_copy` and `rotate_copy`, however parallel range algorithms return a different iterator (as explained above) even if the output size is sufficient.

— we envision serial range algorithms with "range-as-the-output" design in the future, where it is appropriate to return both the *stop point* and `last`, if it was calculated.

To elaborate more on serial range algorithms: the current strategy for the vast majority of them is returning as much calculated data as possible. For example, `ranges::for_each` returns the iterator equal to `last` because that is what the algorithm calculated besides applying a callable object element-wise. This is useful information because users might not have such an iterator before the algorithm call; they might only have a sentinel.

Furthermore, in the future we expect serial range algorithms with "range-as-the-output" that support not only `random_access_iterator`. For such algorithms it totally makes sense to return both the input *stop point* and the input `last`, if the latter was calculated. For the vast majority of algorithms with the output, the input `last` either cannot be calculated (when the output size is insufficient) or matches the stop point. For `reverse_copy` and `rotate_copy` this is not true. We need to keep that in mind when designing these algorithms, because it would be very unfortunate if serial "range-as-the-output" algorithms end up returning something different from parallel ones. Thus, the design of the parallel range algorithms needs to accept that they should return more information for `random_access_<iterator|range>` than seems necessary.

One of the design goals for [P3179R9] is to keep the same return type for the proposed algorithms compared to existing range algorithms, where possible. However, there is a clear indication that we need to do something different for `reverse_copy` and `rotate_copy`.

§ 2 `rotate_copy` recommended design

In [P3179R9] `rotate_copy` returns the iterator past the last copied element for the input. By design, it never returns `last`, and it returns `middle` for two scenarios:

— the output size is sufficient to copy everything from the input.

— the output range is empty.

The raised concern is that users might be surprised at runtime by a completely different return value after switching to parallel execution. Moreover, the actual reason for the different value is not an execution policy but the way how the output range is passed in, which might be viewed as a purely syntactical modification. Consider the following example:

Variations of `rotate_copy`

iterator-as-the-output

```
std::list in{1,2,3,4,5,6};
std::vector<int> out(6);

// in_view is not common_range
auto in_view = std::views::counted(in.begin(), 6);

static_assert(!std::ranges::common_range<decltype(in_view)>);

auto middle = in_view.begin();
std::ranges::advance(middle, 3);

// iterator as the output
auto res = std::ranges::rotate_copy(in_view, middle,
        out.begin());

// res.in compares equal to in_view.end()

auto val = std::reduce(in_view.begin(), res.in);

// val is 21
```

range-as-the-output

```
std::list in{1,2,3,4,5,6};
std::vector<int> out(6);

// in_view is not common_range
auto in_view = std::views::counted(in.begin(), 6);

static_assert(!std::ranges::common_range<decltype(in_view)>);

auto middle = in_view.begin();
std::ranges::advance(middle, 3);

// range as the output
auto res = std::ranges::rotate_copy(in_view, middle, out);

// res.in equals to middle

auto val = std::reduce(in_view.begin(), res.in);

// val is 6
```

Please note, that in the table above we don't use algorithm overloads with execution policy. Instead, we show the future serial range algorithms with "range-as-the-output". That's because we want to use a non-`common_range`, non-`random_access_range`, so that calculating `last` would make sense. Parallel range algorithms require `random_access_range`, so the code would fail to compile if written with execution policy.

While we could address the concern for `rotate_copy` if we jump to the end to return `last` when the output size is sufficient, there would be two problems with that approach:

— it is inconsistent with other "range-as-the-output" algorithms which return the stop point iterator.

— we may calculate `last` as the iterator but still not return it, if the output size is insufficient. See Introduction for more information.

So, if we don't want to surprise users with the different behavior of parallel `rotate_copy` at runtime, we need to make the code fail to compile when the returned value for the existing `rotate_copy` is stored and used. We think that users likely store the return value from range algorithms as `auto` type, so the names of the publicly accessible fields should also be different for the result type of the "range-as-the-output" `rotate_copy`.

Since `rotate_copy` "swaps" two subranges - from `middle` to `last` and from `first` to `middle` - we want to tell how far the algorithm progressed for both of them, thus we propose to change the result type to "alias-ed" `ranges::in_in_out_result`. Its first data member, `in1`, contains one past the last processed iterator for `[middle, last)`, which is either the stop point or equals to `last` if this subrange was fully copied. Similarly, `in2` contains one past the last processed iterator for `[first, middle)`, which might equal to `first` if the processing has not reached this subrange, otherwise it is the stop point up to and including `middle`.

We could introduce some new structure because currently `in_in_out_result` is only used for the algorithms with two input ranges. However, we don't see a reason to introduce yet another type since we think that `in_in_out_result` is perfectly applicable for the discovered use case.

The updated signatures for parallel `ranges::rotate_copy` are:

```
template <class In, class Out>
using rotate_copy_truncated_result = std::ranges::in_in_out_result<In, In, Out>;

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         random_access_iterator O, sized_sentinel_for<O> OutS>
  requires indirectly_copyable<I, O>
  ranges::rotate_copy_truncated_result<I, O>
    ranges::rotate_copy(Ep&& exec, I first, I middle, S last, O result, OutS result_last);

template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
  requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
  ranges::rotate_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::rotate_copy(Ep&& exec, R&& r, iterator_t<R> middle, OutR&& result_r);
```

In our opinion, the design above addresses all the concerns because:

— in `in_out_result` the data member for input is named `in` , while in `in_in_out_result` the names for input are `in1` and `in2`, so we achieve the goal for the code to fail to compile when one uses the output and switches to the parallel overload, even if users store the result as `auto` type (but see a caveat below).

— it takes into account future serial range algorithms with "range-as-the-output", for which `rotate_copy` returns

  — both `last`, calculated as an iterator, and `middle` as the stop point, when the output size is sufficient,

  — the stop points in both subranges (from `middle` to `last` and from `first` to `middle`) when the output size is less than the input one.

A possible implementation of the envisioned serial algorithm:

```
struct rotate_copy_fn
{
    template<std::forward_iterator I, std::sentinel_for<I> S, std::forward_iterator O, std::sentinel_for<O> OutS>
    requires std::indirectly_copyable<I, O>
    constexpr std::ranges::rotate_copy_truncated_result<I, O>
        operator()(I first, I middle, S last, O result, OutS result_last) const
    {
        while (middle != last && result != result_last)
        {
            *result = *middle;
            ++middle;
            ++result;
        }

        while (first != middle && result != result_last)
        {
            *result = *first;
            ++first;
            ++result;
        }
        return {std::move(middle), std::move(first), std::move(result)};
    }

    template<std::ranges::forward_range R, std::ranges::forward_range OutR>
    requires std::indirectly_copyable<std::ranges::iterator_t<R>, std::ranges::iterator_t<OutR>>
```

```
    constexpr std::ranges::rotate_copy_truncated_result<std::ranges::borrowed_iterator_t<R>, std::ranges::borrowed_iter-
        ator_t<OutR>>
        operator()(R&& r, std::ranges::iterator_t<R> middle, OutR&& result_r) const
    {
        return (*this)(std::ranges::begin(r), std::move(middle),
                       std::ranges::end(r), std::ranges::begin(result_r), std::ranges::end(result_r));
    }
};

inline constexpr rotate_copy_fn rotate_copy{};
```

Please note that we propose the following order for returned iterators:

— `in1` as a stop point in `[middle, last]`.

— `in2` as a stop point in `[first, middle]`.

Even though both belong to the same input range, those are just stop points in two subranges, so we don't want users to think of them as of a valid `std::ranges::subrange`.

Caveat: Since C++26 we can imagine a code that compiles for both serial range algorithms and the parallel ones even with the proposed design in this paper, because of structured binding with a pack ([P1061R10]).

Let's compare:

Store the `rotate_copy` result to a structured binding with a pack

Serial `ranges::rotate_copy`	Parallel `ranges::rotate_copy` with the proposed design

`template <typename R, typename OutR>`
`void foo(R&& in, OutR&& out) {`
`  auto middle = std::ranges::begin(in);`
`  std::advance(middle, 3);`
`  // possibly to ignore rest...`
`  auto [in_last, ...rest] =`
`  std::ranges::rotate_copy(in, middle,`
`  std::ranges::begin(out));`
`  // sizeof...(rest) == 1`
`  // in_last equals to ranges::end(in)`
`}`
	`template <typename R, typename OutR>`
	`void foo(R&& in, OutR&& out) {`
	`  auto middle = std::ranges::begin(in);`
	`  std::advance(middle, 3);`
	`  // possibly to ignore rest...`
	`  auto [in_last, ...rest] =`
	`  std::ranges::rotate_copy(std::execution::par, in,`
	`  middle, out);`
	`  // with this paper, sizeof...(rest) == 2`
	`  // in_last equals to ranges::end(in) if the output size`
	`  is sufficient`
	`}`

We can imagine such code if people want to only store the first or the last data member of the returned object and ignore the rest. With the code shown above it's even more important that `in1` is a stop point in `[middle, last]` because, assuming that the size of `out` is sufficient for the whole input, this code compiles and works at runtime as expected. The same can be said if the structured binding is written as `auto [...rest, out_last]` to get the iterator to the output.

§ 3 `reverse_copy` recommended design

For `reverse_copy` the proposed algorithm in [P3179R9] always returns the iterator to the last copied element (in the reversed order) for the input. There is no "dual meaning" for that returned iterator. In isolation it seems perfectly fine, however two concerns come in mind:

— different behavior at runtime compared to the serial counterpart (which always returns `last`), when the output size is sufficient.

— for the envisioned serial `ranges::reverse_copy` with "range-as-the-output" we would calculate `last` as the iterator but not return that. See Introduction for more information.

Consider the following example:

Variations of `reverse_copy`

iterator-as-the-output `reverse_copy`	range-as-the-output `reverse_copy`

`std::list in{1,2,3,4,5,6};`
`std::vector<int> out(6);`
`// in_view is not common_range`
`auto in_view = std::views::counted(in.begin(), 6);`
`static_assert(!std::ranges::common_range<decltype(in_view)>);`
`// iterator as the output`
`auto res = std::ranges::reverse_copy(in_view, out.begin());`
	`std::list in{1,2,3,4,5,6};`
	`std::vector<int> out(6);`
	`// in_view is not common_range`
	`auto in_view = std::views::counted(in.begin(), 6);`
	`static_assert(!std::ranges::common_range<decltype(in_view)>);`
	`// range as the output`
	`auto res = std::ranges::reverse_copy(in_view, out);`

iterator-as-the-output `reverse_copy`	range-as-the-output `reverse_copy`

`// res.in compares equal to in_view.end()`
 	
`auto val = std::reduce(view.begin(), res.in);`
 	
`// val is 21`
	`// res.in equals to in_view.begin()`
	
	`auto val = std::reduce(view.begin(), res.in);`
	
	`// val is 0`

Essentially, the story is similar with `rotate_copy`. All the following ruminations also apply for parallel `reverse_copy`:

— a new return type vs `in_in_out_result`

— returning two different points in the input range rather than a valid `std::ranges::subrange`

— use of structured binding with a parameter pack

The solution we see is also similar. We propose to return "alias-ed" `ranges::in_in_out_result`:

— `in1` is always the iterator equal to `last`.

— `in2` represents the stop point, as proposed in [P3179R9].

The updated signatures for parallel `ranges::reverse_copy` are:

```
template <class In, class Out>
using reverse_copy_truncated_result = std::ranges::in_in_out_result<In, In, Out>;

template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         random_access_iterator O, sized_sentinel_for<O> OutS>
  requires indirectly_copyable<I, O>
  ranges::reverse_copy_truncated_result<I, O>
    ranges::reverse_copy(Ep&& exec, I first, S last, O result, OutS result_last);

template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
  requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
  ranges::reverse_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::reverse_copy(Ep&& exec, R&& r, OutR&& result_r);
```

A possible implementation of the envisioned serial algorithm:

```
struct reverse_copy_fn
{
  template<std::bidirectional_iterator I, std::sentinel_for<I> S, std::forward_iterator O, std::sentinel_for<O> OutS>
    requires std::indirectly_copyable<I, O>
    constexpr std::ranges::reverse_copy_truncated_result<I, O>
        operator()(I first, S last, O result, OutS result_last) const
    {
        auto last_iter = std::ranges::next(first, last);
        auto pos = last_iter;

        while (pos != first && result != result_last)
        {
            *result = *--pos;
            ++result;
        }

        return {last_iter, pos, result};
    }

    template<std::ranges::bidirectional_range R, std::ranges::forward_range OutR>
    requires std::indirectly_copyable<std::ranges::iterator_t<R>, std::ranges::iterator_t<OutR>>
     constexpr std::ranges::reverse_copy_truncated_result<std::ranges::borrowed_iterator_t<R>, std::ranges::borrowed_it-
        erator_t<OutR>>
        operator()(R&& r, OutR&& result_r) const
    {
        return (*this)(std::ranges::begin(r), std::ranges::end(r),
                       std::ranges::begin(result_r), std::ranges::end(result_r));
    }
};

inline constexpr reverse_copy_fn reverse_copy{};
```

§ 4 Return types naming consideration

The current design for `rotate_copy` returns the stop points for two subranges - `[middle, last)` and `[first, middle)` - in the input. It means that:

— for the empty output it returns `{middle, first, out}`

— for the output that is sufficient to take some elements from `[middle, last)` it returns `{stop_point, first, out}`

— for the output that is sufficient to take all elements from `[middle, last)` it returns `{last, first, out}`

— for the output that is sufficient to take all elements from `[middle, last)` and some elements from `[first, middle)` it returns `{last, stop_point, out}`

— for the output that is sufficient to take all elements from `[middle, last)` and all elements from `[first, middle)` it returns `{last, middle, out}`

There is a specific reason why the design is like that. The users can easily figure out what elements were not copied from the input without extra checks. In the current design with the names `in1` and `in2` those uncopied elements are `[in1, last)` and `[in2, middle)`.

Aor `rotate_copy` the alternative names to `in1` and `in2` could be:

— `stop1` and `stop2`

— `stop_right` and `stop_left`

— `stage1` and `stage2`

— maybe something else along those lines

Those names make sense for the current design but are they anyhow better than `in1` and `in2`? In our opinion - not really. Furthermore, to use them we need to introduce a new type to C++ standard library, which does not give any more clarity from our perspective.

To conclude, we don't see a good alternative, compared to what is already proposed in the paper, in terms of public data member names of `rotate_copy` return type.

For `reverse_copy` the reasonably good alternatives for `in1` and `in2` could be:

— `last` and `stop`

— `in` and `stop`

— `last` and `in`

Then the question is do we really want to introduce a new type to the C++ standard library for the sake of one algorithm only? Our recommendation is that we should not do it.

To summarize, we believe that `in_in_out_result` is good enough type to be used that implies `in1` and `in2` names for public data members.

§ 5 Alternative design

There are the following alternatives to the proposed design:

— Keep the [P3179R9] status quo. The concerns are:

  — The different behavior at runtime, when switching the code to parallel algorithms.

  — No calculated `last` for future serial range algorithms with "range-as-the-output".

— Keep `reverse_copy` as proposed in [P3179R9], return `last` for parallel `rotate_copy` when the output is sufficient. The concerns are:

  — for `reverse_copy` the concerns from the previous bullets apply.

  — `rotate_copy` becomes inconsistent with other parallel range algorithms with "range-as-the-output" because it can "jump" to `last` instead of returning the stop point.

  — The parallel `rotate_copy` calculates `last` but does not return it, so it likely means a different return type for serial "range-as-the-output" `rotate_copy` in the future.

We think these alternatives would be worse. If this proposal does not have a consensus, authors' preference is to remove parallel `reverse_copy` and `rotate_copy` from [P3179R9] and add them later, possibly together with the serial "range-as-the-output" algorithms.

§ 6 Wording

The diff shown below is against [P3179R9].

§ 6.1 Modify [algorithm.syn]

```
namespace ranges {
  template<class I, class O>
    using reverse_copy_result = in_out_result<I, O>;
+  template<class I, class O>
+    using reverse_copy_truncated_result = in_in_out_result<I, I, O>;

  template<bidirectional_iterator I, sentinel_for<I> S, weakly_incrementable O>
    requires indirectly_copyable<I, O>
    constexpr reverse_copy_result<I, O>
      reverse_copy(I first, S last, O result);
  template<bidirectional_range R, weakly_incrementable O>
    requires indirectly_copyable<iterator_t<R>, O>
    constexpr reverse_copy_result<borrowed_iterator_t<R>, O>
```

```
      reverse_copy(R&& r, O result);

  template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
           random_access_iterator O, sized_sentinel_for<O> OutS>
    requires indirectly_copyable<I, O>
-    reverse_copy_result<I, O>
+    reverse_copy_truncated_result<I, O>
      reverse_copy(Ep&& exec, I first, S last, O result, OutS result_last);    // freestanding-deleted
  template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
    requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
-    reverse_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
+    reverse_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
      reverse_copy(Ep&& exec, R&& r, OutR&& result_r);    // freestanding-deleted
}

namespace ranges {
  template<class I, class O>
    using rotate_copy_result = in_out_result<I, O>;
+  template<class I, class O>
+    using rotate_copy_truncated_result = in_in_out_result<I, I, O>;

  template<forward_iterator I, sentinel_for<I> S, weakly_incrementable O>
    requires indirectly_copyable<I, O>
    constexpr rotate_copy_result<I, O>
      rotate_copy(I first, I middle, S last, O result);
  template<forward_range R, weakly_incrementable O>
    requires indirectly_copyable<iterator_t<R>, O>
    constexpr rotate_copy_result<borrowed_iterator_t<R>, O>
      rotate_copy(R&& r, iterator_t<R> middle, O result);

  template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
           random_access_iterator O, sized_sentinel_for<O> OutS>
    requires indirectly_copyable<I, O>
-    ranges::rotate_copy_result<I, O>
+    ranges::rotate_copy_truncated_result<I, O>
      ranges::rotate_copy(Ep&& exec, I first, I middle, S last, O result, OutS result_last);    // freestanding-deleted
  template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
    requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
-    ranges::rotate_copy_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
+    ranges::rotate_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
      ranges::rotate_copy(Ep&& exec, R&& r, iterator_t<R> middle, OutR&& result_r);    // freestanding-deleted
}
```

§ 6.2 Modify [alg.reverse]

```
template<bidirectional_iterator I, sentinel_for<I> S, weakly_incrementable O>
  requires indirectly_copyable<I, O>
  constexpr ranges::reverse_copy_result<I, O>
    ranges::reverse_copy(I first, S last, O result);
template<bidirectional_range R, weakly_incrementable O>
  requires indirectly_copyable<iterator_t<R>, O>
  constexpr ranges::reverse_copy_result<borrowed_iterator_t<R>, O>
    ranges::reverse_copy(R&& r, O result);
```

5 Let *N* be `last - first`.

6 *Preconditions*: The ranges `[first, last)` and `[result, result + N)` do not overlap.

7 *Effects*: Copies the range `[first, last)` to the range `[result, result + N)` such that for every non-negative integer `i < N` the following assignment takes place: `*(result + N - 1 - i) = *(first + i)`.

8 *Returns*:

(8.1) — `result + N` for the overloads in namespace `std`.

(8.2) — `{last, result + N}` for the overloads in namespace `ranges`.

9 *Complexity*: Exactly *N* assignments.

```
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         random_access_iterator O, sized_sentinel_for<O> OutS>
  requires indirectly_copyable<I, O>
  ranges::reverse_copy_truncated_result<I, O>
    ranges::reverse_copy(Ep&& exec, I first, S last, O result, OutS result_last);
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
  requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
  ranges::reverse_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::reverse_copy(Ep&& exec, R&& r, OutR&& result_r);
```

x Let *N* be min(`last - first`, `result_last - result`), and let *NEW_FIRST* be `first + (last - first) - N`.

x *Preconditions*: The ranges `[first, last)` and `[result, result + N)` do not overlap.

x *Effects*: Copies the range `[NEW_FIRST, last)` to the range `[result, result + N)` such that for every non-negative integer `i < N` the following assignment takes place: `*(result + N - 1 - i) = *(NEW_FIRST + i)`.

x *Returns*: `{last, NEW_FIRST, result + N}`.

~~[*Note:* While the return type for the parallel and non-parallel algorithm overloads in the namespace `ranges` is the same the semantics is different because the parallel range algorithm overloads `result_last = result` can be insufficient to copy all data from the input. — *end note*]~~

x *Complexity*: Exactly `N` assignments.

§ 6.3 Modify [alg.rotate]

```
template<forward_iterator I, sentinel_for<I> S, weakly_incrementable O>
    requires indirectly_copyable<I, O>
    constexpr ranges::rotate_copy_result<I, O>
      ranges::rotate_copy(I first, I middle, S last, O result);
```

6 Let `N` be `last - first`.

7 *Preconditions*: `[first, middle)` and `[middle, last)` are valid ranges. The ranges `[first, last)` and `[result, result + N)` do not overlap.

8 *Effects*: Copies the range `[first, last)` to the range `[result, result + N)` such that for each non-negative integer `i < N` the following assignment takes place: `*(result + i) = *(first + (i + (middle - first)) % N)`.

9 *Returns*:

(9.1) — `result + N` for the overloads in namespace `std`.

(9.2) — `{last, result + N}` for the overload in namespace `ranges`.

10 *Complexity*: Exactly `N` assignments.

```
template<execution-policy Ep, random_access_iterator I, sized_sentinel_for<I> S,
         random_access_iterator O, sized_sentinel_for<O> OutS>
  requires indirectly_copyable<I, O>
  ranges::rotate_copy_truncated_result<I, O>
    ranges::rotate_copy(Ep&& exec, I first, I middle, S last, O result, OutS result_last);
```

x Let `M` be `last - first` and `N` be min(`M`, `result_last - result`).

x *Preconditions*: `[first, middle)` and `[middle, last)` are valid ranges. The ranges `[first, last)` and `[result, result + N)` do not overlap.

x *Effects*: Copies the range `[first, last)` to the range `[result, result + N)` such that for each non-negative integer `i < N` the following assignment takes place: `*(result + i) = *(first + (i + (middle - first)) % M)`.

x *Returns*: ~~`{first + (N + (middle - first)) % M, result + N}`~~.

(x.1) — `{middle + N, first, result + N}` if `N` is less than `last - middle`.

(x.2) — Otherwise, `{last, first + (N + (middle - first)) % M, result + N}`.

~~[*Note:* While the return type for the parallel and non-parallel algorithm overloads in the namespace `ranges` is the same the semantics is different because the parallel range algorithm overloads `result_last = result` can be insufficient to copy all data from the input. — *end note*]~~

x *Complexity*: Exactly `N` assignments.

```
template<forward_range R, weakly_incrementable O>
  requires indirectly_copyable<iterator_t<R>, O>
  constexpr ranges::rotate_copy_result<borrowed_iterator_t<R>, O>
    ranges::rotate_copy(R&& r, iterator_t<R> middle, O result);
```

Effects: Equivalent to: `return ranges::rotate_copy(ranges::begin(r), middle, ranges::end(r), std::move(result));`

```
template<execution-policy Ep, sized-random-access-range R, sized-random-access-range OutR>
  requires indirectly_copyable<iterator_t<R>, iterator_t<OutR>>
  ranges::rotate_copy_truncated_result<borrowed_iterator_t<R>, borrowed_iterator_t<OutR>>
    ranges::rotate_copy(Ep&& exec, R&& r, iterator_t<R> middle, OutR&& result_r);
```

Effects: Equivalent to: `return ranges::rotate_copy(std::forward<Ep>(exec), ranges::begin(r), middle, ranges::end(r), ranges::begin(result_r), ranges::end(result_r));`

§ 7 Revision history

§ 7.1 R1 => R2

— Add naming considerations for public data member of `reverse_copy` and `rotate_copy` return types.

§ 7.2 R0 => R1

— Editorial changes and fixes

— Add the formal wording

§ 8 Polls

§ 8.1 SG9, Sofia, 2025

POLL: We want to change the return type of the parallel `std::ranges::rotate_copy` to return both “stop” and “end”, instead of just “stop” (unlike the serial one where it is “end”).

SF	F	N	A	SA
5	3	1	0	0

POLL: We want to change the return type of the parallel `std::ranges::reverse_copy` to return both “stop” and “end”, instead of just “stop” (unlike the serial one where it is “end”).

SF	F	N	A	SA
4	4	1	0	0

POLL: We want to use `std::in_in_out_result` as the return type of `std::ranges::rotate_copy` and `std::ranges::reverse_copy` (as proposed in the paper).

SF	F	N	A	SA
1	2	4	2	1

POLL: We want to use a new return type with members e.g. `{in_end, in_stop, out}` (names TBD) instead of `std::in_in_out_result`

SF	F	N	A	SA
1	3	5	0	0

POLL: Remove the parallel execution overloads of `std::ranges::rotate_copy` and `std::ranges::reverse_copy` from P3179R9 scheduled for C++26.

SF	F	N	A	SA
3	2	1	3	0

POLL: Forward P3709R1, with the return type change guidance from above to LEWG for inclusion in C++26.

SF	F	N	A	SA
2	6	0	1	1

§ 8.2 LEWG, Sofia, 2025

POLL: We want to change the return type of “rotate_copy” and “reverse_copy” to something similar to in_in_out

SF	F	N	A	SA
3	7	2	0	0

POLL: In P3709R1 rotate_copy and reverse_copy should have a dedicated return type instead of re-using `in_in_out_result`

SF	F	N	A	SA
7	5	2	2	0

§ 9 Acknowledgements

— Thanks to Jonathan Mueller for a fruitful discussion on `reverse_copy` and `rotate_copy` return type.

§ 10 References

[P1061R10] Barry Revzin, Jonathan Wakely. 2024-11-24. Structured Bindings can introduce a Pack.
https://wg21.link/p1061r10

[P3179R9] Ruslan Arutyunyan, Alexey Kukanov, and Bryce Adelstein Lelbach. C++ parallel range algorithms. https://isocpp.org/files/papers/P3179R9.html